Cab Booking System for Data Analysis (MySQL)

Project Overview:

The project simulates a real-world cab booking system to demonstrate how structured databases are used to store, manage, and analyze operational data. It involves creating relational tables, inserting meaningful records, and executing SQL queries to uncover insights on customer behavior, driver performance, trip patterns, and revenue trends

Dataset Summary:

The dataset represents a simulated cab booking system containing six interconnected tables that capture end-to-end ride information. It includes customer details, driver profiles, cab information, booking records, trip-level metrics, and customer feedback. Each table is linked through primary and foreign keys, enabling detailed analysis of booking trends, trip performance, cancellations, driver efficiency, and customer satisfaction. The dataset is designed to support a wide range of SQL-based analytical queries for understanding operational, behavioral, and revenue-related insights.

Setting up the Database Schema.

1. Creating Database 'cb' and using it.

```
1     -- CREATING AND USING DATABASE 'cb'
2 •   create database cb;
3 •   use cb;
```

2. Customer table.

```
5     -- Creating Customer table
6 •   create table Customers(
7         CustomerID INT PRIMARY KEY,
8         Name VARCHAR(100),
9         Email VARCHAR(100),
10        RegistrationDate DATE
11    );
```

3. Drivers table.

```
13      -- Creating Drivers table
14  •   create table Drivers(
15          DriverID INT PRIMARY KEY,
16          Name VARCHAR(100),
17          JoinDate DATE
18      );
```

4. Cabs table.

```
20      -- Creating Cabs table
21  •   create table Cabs(
22          CabID INT PRIMARY KEY,
23          DriverID INT,
24          VehicleType VARCHAR(20),
25          PlateNumber VARCHAR(20),
26          FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID)
27      );
```

5. Bookings table

```
29      -- Creating Bookings table
30  •   create table Bookings(
31          BookingID INT PRIMARY KEY,
32          CustomerID INT,
33          CabID INT,
34          BookingDate DATETIME,
35          Status VARCHAR(20),
36          PickupLocation VARCHAR(100),
37          DropoffLocation VARCHAR(100),
38          FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
39          FOREIGN KEY (CabID) REFERENCES Cabs(CabID)
40      );
```

6. TripDetails table.

```
42      -- Creating TripDetails table
43  •   create table TripDetails(
44          TripID INT PRIMARY KEY,
45          BookingID INT,
46          StartTime DATETIME,
47          EndTime DATETIME,
48          DistanceKM FLOAT,
49          Fare FLOAT,
50          FOREIGN KEY (BookingID) REFERENCES Bookings(BookingID)
51      );
```

7. FeedBack table.

```
53      -- Creating Feedback table
54  •   create table Feedback(
55          FeedbackID INT PRIMARY KEY,
56          BookingID INT,
57          Rating FLOAT,
58          Comments TEXT,
59          FeedbackDate DATE,
60          FOREIGN KEY (BookingID) REFERENCES Bookings(BookingID)
61      );
```

- **Data Insertion**

  1. Insert Customers

```
63      -- Inserting values into Customers table
64  •   INSERT INTO Customers (CustomerID, Name, Email, RegistrationDate)
65      VALUES (1, 'Alice Johnson', 'alice@example.com', '2023-01-15'),
66      (2, 'Bob Smith', 'bob@example.com', '2023-02-20'),
67      (3, 'Charlie Brown', 'charlie@example.com', '2023-03-05'),
68      (4, 'Diana Prince', 'diana@example.com', '2023-04-10'),
69      (5, 'Mickey Mouse', 'mickey@example.com', '2025-04-15'),
70      (6, 'Karan Kumar', 'karan@example.com', '2025-05-03'),
71      (7, 'Avinash Ray', 'avinash@example.com', '2025-05-10'),
72      (8, 'Anjali Kumari', 'anjali@example.com', '2025-05-23'),
73      (9, 'Anita Ray', 'anita@example.com', '2025-06-12'),
74      (10, 'Awadh Kishor', 'awadh@example.com', '2025-06-18');
```

2. Insert Drivers

```
76      -- Inserting values into Drivers table
77 •    INSERT INTO Drivers (DriverID, Name, JoinDate)
78      VALUES (101, 'John Driver', '2022-05-10'),
79      (102, 'Linda Miles', '2022-07-25'),
80      (103, 'Kevin Road', '2023-01-01'),
81      (104, 'Sandra Swift', '2022-11-11'),
82      (105, 'Devendra Jacks','2021-08-14'),
83      (106, 'Binod Bhatade', '2022-10-21'),
84      (107, 'Tejas Kamble', '2023-06-17'),
85      (108, 'Rahul Kadam', '2022-04-13'),
86      (109, 'Gaurav Sawant', '2022-05-20'),
87      (110, 'Sachin Bane', '2021-03-18');
```

3. Insert Cabs

```
89      -- Inserting values into Cabs table
90 •    INSERT INTO Cabs (CabID, DriverID, VehicleType, PlateNumber)
91      VALUES (1001, 101, 'Sedan', 'ABC1234'),
92      (1002, 102, 'SUV', 'XYZ5678'),
93      (1003, 103, 'Sedan', 'LMN8901'),
94      (1004, 104, 'SUV', 'PQR3456'),
95      (1005, 105, 'Toyota', 'ADS2435'),
96      (1006, 106, 'Hyundai', 'KHJ3672'),
97      (1007, 107, 'Sedan', 'GHF6752'),
98      (1008, 108, 'Toyota', 'YUH5623'),
99      (1009, 109, 'Hyundai', 'JGF7823'),
100     (1010, 110, 'SUV', 'KYD9834');
```

4. Insert Bookings

```sql
-- Inserting values into Bookings tables
INSERT INTO Bookings (BookingID, CustomerID, CabID, BookingDate, Status, PickupLocation, DropoffLocation)
VALUES (201, 1, 1001, '2024-10-01 08:30:00', 'Completed', 'Downtown', 'Airport'),
(202, 2, 1002, '2024-10-02 09:00:00', 'Completed', 'Mall', 'University'),
(203, 3, 1003, '2024-10-03 10:15:00', 'Canceled', 'Station', 'Downtown'),
(204, 4, 1004, '2024-10-04 14:00:00', 'Completed', 'Suburbs', 'Downtown'),
(205, 1, 1002, '2024-10-05 18:45:00', 'Completed', 'Downtown', 'Airport'),
(206, 2, 1001, '2024-10-06 07:20:00', 'Canceled', 'University', 'Mall'),
(207, 3, 1003, '2024-10-07 11:10:00', 'Completed', 'Mall', 'Airport'),
(208, 4, 1004, '2024-10-08 12:00:00', 'Completed', 'Downtown', 'Suburbs'),
(209, 1, 1001, '2024-10-09 15:30:00', 'Canceled', 'University', 'Station'),
(210, 2, 1002, '2024-10-10 17:45:00', 'Completed', 'Station', 'Downtown'),
(211, 3, 1003, '2024-10-11 08:50:00', 'Completed', 'Mall', 'University'),
(212, 4, 1004, '2024-10-12 14:20:00', 'Completed', 'Suburbs', 'Airport'),
(213, 1, 1002, '2024-10-13 09:15:00', 'Canceled', 'Downtown', 'Station'),
(214, 2, 1001, '2024-10-14 13:40:00', 'Completed', 'University', 'Mall'),
(215, 3, 1003, '2024-10-15 16:05:00', 'Completed', 'Station', 'Downtown'),
(216, 4, 1004, '2024-10-16 10:25:00', 'Completed', 'Mall', 'Airport'),
(217, 1, 1001, '2024-10-17 18:55:00', 'Completed', 'Downtown', 'University'),
(218, 2, 1002, '2024-10-18 07:35:00', 'Canceled', 'Suburbs', 'Downtown'),
(219, 3, 1003, '2024-10-19 11:45:00', 'Completed', 'Airport', 'Mall'),
(220, 4, 1004, '2024-10-20 12:30:00', 'Completed', 'Station', 'Suburbs'),
(221, 1, 1002, '2024-10-21 14:15:00', 'Completed', 'Mall', 'Airport'),
(222, 2, 1003, '2024-10-22 16:50:00', 'Canceled', 'University', 'Station'),
(223, 3, 1001, '2024-10-23 09:05:00', 'Completed', 'Downtown', 'Mall'),
(224, 4, 1004, '2024-10-24 10:55:00', 'Completed', 'Suburbs', 'Airport'),
(225, 1, 1001, '2024-10-25 18:10:00', 'Completed', 'Downtown', 'Suburbs'),
(226, 2, 1002, '2024-10-26 08:40:00', 'Canceled', 'Mall', 'Downtown'),
(227, 2, 1005, '2024-10-27 09:25:00', 'Completed', 'Mall', 'Airport'),
```

5. Insert TripDetails

```sql
-- Inserting values into TripDetails table
INSERT INTO TripDetails (TripID, BookingID, StartTime, EndTime, DistanceKM, Fare)
VALUES (301, 201, '2024-10-01 08:45:00', '2024-10-01 09:20:00', 18.5, 250.00),
(302, 202, '2024-10-02 09:10:00', '2024-10-02 09:40:00', 12.0, 180.00),
(303, 204, '2024-10-04 14:10:00', '2024-10-04 14:40:00', 10.0, 150.00),
(304, 205, '2024-10-05 18:50:00', '2024-10-05 19:30:00', 20.0, 270.00),
(305, 206, '2024-10-06 07:30:00', '2024-10-06 08:05:00', 14.0, 200.00),
(306, 207, '2024-10-07 11:20:00', '2024-10-07 11:55:00', 16.5, 230.00),
(307, 208, '2024-10-08 12:10:00', '2024-10-08 12:45:00', 13.0, 190.00),
(308, 209, '2024-10-09 15:40:00', '2024-10-09 16:15:00', 17.0, 240.00),
(309, 210, '2024-10-10 17:55:00', '2024-10-10 18:25:00', 11.0, 160.00),
(310, 211, '2024-10-11 09:00:00', '2024-10-11 09:30:00', 12.5, 175.00),
(311, 212, '2024-10-12 14:30:00', '2024-10-12 15:10:00', 19.0, 260.00),
(312, 213, '2024-10-13 09:25:00', '2024-10-13 10:00:00', 15.0, 210.00),
(313, 214, '2024-10-14 13:50:00', '2024-10-14 14:20:00', 9.5, 140.00),
(314, 215, '2024-10-15 16:15:00', '2024-10-15 16:50:00', 18.0, 245.00),
(315, 216, '2024-10-16 10:35:00', '2024-10-16 11:05:00', 10.5, 155.00),
(316, 217, '2024-10-17 19:05:00', '2024-10-17 19:45:00', 21.0, 290.00),
(317, 218, '2024-10-18 07:45:00', '2024-10-18 08:20:00', 13.5, 185.00),
(318, 219, '2024-10-19 11:55:00', '2024-10-19 12:30:00', 14.8, 205.00),
(319, 220, '2024-10-20 12:40:00', '2024-10-20 13:15:00', 16.2, 225.00);
```

6. Insert Feedback

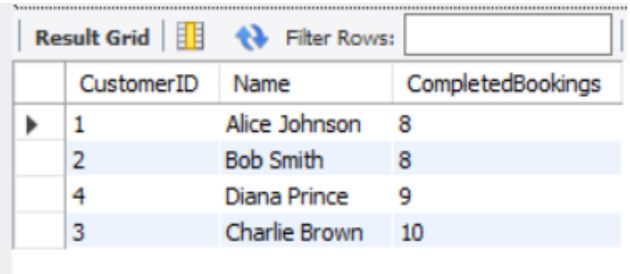
```sql
-- Inserting into Feedback table
INSERT INTO Feedback (FeedbackID, BookingID, Rating, Comments, FeedbackDate)
VALUES (401, 201, 4.5, 'Smooth ride', '2024-10-01'),
(402, 202, 3.0, 'Driver was late', '2024-10-02'),
(403, 204, 5.0, 'Excellent service', '2024-10-04'),
(404, 205, 2.5, 'Cab was not clean', '2024-10-05'),
(405, 204, 4.0, 'Good ride', '2024-10-06'),
(406, 207, 3.5, 'Average experience', '2024-10-07'),
(407, 210, 5.0, 'Very professional driver', '2024-10-08'),
(408, 213, 2.0, 'Driver took long route', '2024-10-09'),
(409, 218, 4.5, 'Smooth and quick trip', '2024-10-10'),
(410, 221, 3.0, 'Could be better', '2024-10-11'),
(411, 226, 4.8, 'Excellent overall', '2024-10-12'),
(412, 209, 1.5, 'Cab was smelly', '2024-10-13'),
(413, 215, 5.0, 'Very comfortable ride', '2024-10-14'),
(414, 220, 3.8, 'Good but slightly delayed', '2024-10-15'),
(415, 212, 2.5, 'Driver was rude', '2024-10-16'),
(416, 224, 4.2, 'Nice and clean cab', '2024-10-17'),
(417, 208, 3.7, 'Decent ride', '2024-10-18'),
(418, 219, 4.9, 'Perfect experience', '2024-10-19'),
(419, 214, 2.0, 'Too slow', '2024-10-20'),
(420, 217, 4.3, 'Friendly driver', '2024-10-21'),
(421, 222, 3.2, 'Satisfactory service', '2024-10-22'),
(422, 225, 5.0, 'Loved the ride', '2024-10-23'),
(423, 211, 2.8, 'Not very clean', '2024-10-24'),
(424, 223, 4.7, 'Very smooth journey', '2024-10-25');
```

• SQL Queries to Explore Insights

Now that the database is set up with records, beginning with answering real-world questions through SQL queries.

1. Write an SQL query to display each customer's ID, name, and the total number of bookings they have completed. Only count bookings where the status is 'Completed'. Sort the result in ascending order based on the number of completed bookings.

```sql
select c.CustomerID, c.Name, count(*) as CompletedBookings from Customers c
join Bookings b on c.CustomerID = b.CustomerID
where b.status = 'Completed'
group by c.CustomerID, c.Name
order by CompletedBookings;
```



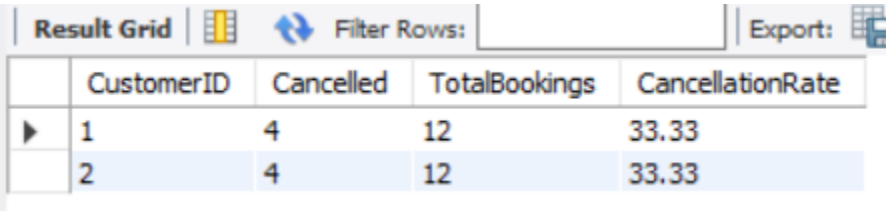
Result Grid | Filter Rows:

CustomerID	Name	CompletedBookings
1	Alice Johnson	8
2	Bob Smith	8
4	Diana Prince	9
3	Charlie Brown	10

Insights: This query helps identify customers with the most or least completed bookings, useful for analyzing customer engagement and loyalty.

2. Write an SQL query to find customers whose booking cancellation rate is more than 20%. The output should display CustomerID, number of cancelled bookings, total bookings, and cancellation percentage.

```
select CustomerID,
    sum(case when status = 'Canceled' then 1 else 0 end) as Cancelled,
    count(*) as TotalBookings,
    round(100.0 * sum(case when status = 'Canceled' then 1 else 0 end ) / count(*), 2) as CancellationRate
from Bookings
group by CustomerID
Having CancellationRate > 20;
```



Result Grid | Filter Rows: | Export:

CustomerID	Cancelled	TotalBookings	CancellationRate
1	4	12	33.33
2	4	12	33.33

Insights: This query helps identify customers who frequently cancel bookings, which may indicate unreliable or inconsistent booking behavior. Such data can help businesses reduce operational risks by monitoring high-cancellation customers.

3. Write an SQL query to find out on which day of the week the highest number of bookings were made. Display each day of the week along with the total number of bookings and sort the results from highest to lowest.

```
select date_format(BookingDate, '%W') as DayofWeek, count(*) as TotalBookings from Bookings
group by date_format(BookingDate, '%W')
order by TotalBookings DESC;
```

Result Grid | Filter Rows:

DayofWeek	TotalBookings
Tuesday	7
Wednesday	7
Thursday	7
Friday	7
Saturday	6
Sunday	6
Monday	6

Insights: This query helps identify peak booking days, which is useful for operational planning and resource allocation.

Understanding which weekdays have the highest bookings allows businesses to optimize staffing, pricing, and promotions.

It also highlights customer behavior trends, helping companies prepare for high-demand days.

4. Write an SQL query to find drivers whose average feedback rating is Below 3.0. Use the Drivers, Cabs, Bookings, and Feedback tables.

```
select d.DriverID, d.name, round(avg(f.Rating),2) as AvgRating from Drivers d
join Cabs c on d.DriverID = c.DriverID
join Bookings b on c.CabID = b.CabID
join Feedback f on b.BookingID = f.BookingID
WHERE f.Rating IS NOT NULL
group by d.DriverID, d.name
having avg(f.Rating) < 3;
```

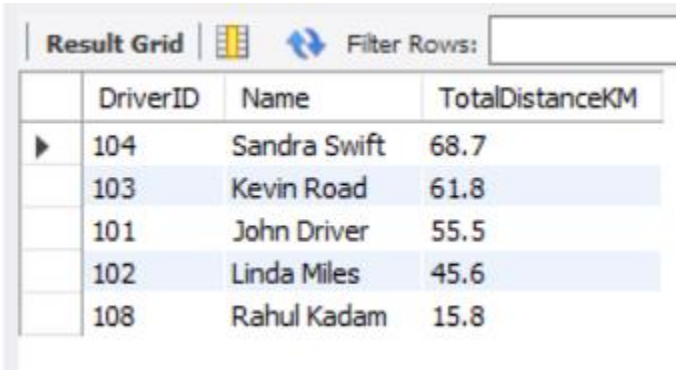
Result Grid | Filter Rows:

DriverID	name	AvgRating
106	Binod Bhatade	2.95
108	Rahul Kadam	2.6
109	Gaurav Sawant	2.8

Insights: This query identifies drivers who may require performance improvement, based on customer feedback. It improves overall service quality by highlighting consistently underperforming drivers.

5. Write an SQL query to find the top 5 drivers who have traveled the highest total distance in completed trips.

```
select d.DriverID, d.Name, round(sum(t.DistanceKM),2) as TotalDistanceKM from Drivers d
join Cabs c on d.DriverID = c.DriverID
join Bookings b on c.CabID = b.CabID
join Tripdetails t on b.BookingID = t.BookingID
where b.Status = 'Completed'
group by d.DriverID, d.Name
order by TotalDistanceKM DESC
LIMIT 5;
```



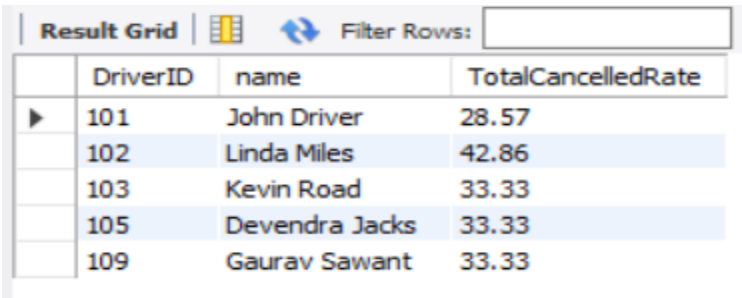
Result Grid | Filter Rows:

DriverID	Name	TotalDistanceKM
104	Sandra Swift	68.7
103	Kevin Road	61.8
101	John Driver	55.5
102	Linda Miles	45.6
108	Rahul Kadam	15.8

Insights: This query helps identify top-performing drivers who cover the longest distances, showing strong activity and productivity.

6. Write an SQL query to find all drivers whose trip cancellation rate is greater than 25%.

```
select d.DriverID, d.name,
    round(sum(case when b.status = 'Canceled' then 1 else 0 end) * 100 / count(*),2) as TotalCancelledRate
from drivers d
join cabs c on d.DriverID = c.DriverID
join Bookings b on c.CabID = b.CabID
group by d.DriverID, d.name
having TotalCancelledRate > 25;
```



Result Grid | Filter Rows:

DriverID	name	TotalCancelledRate
101	John Driver	28.57
102	Linda Miles	42.86
103	Kevin Road	33.33
105	Devendra Jacks	33.33
109	Gaurav Sawant	33.33

Insights: This query identifies drivers with unusually high cancellation behavior, which may affect customer experience and platform reliability. It supports decision-making for driver monitoring, retraining, or policy adjustments to improve service quality.

7. Write an SQL query to calculate the monthly revenue generated from all completed trips.

```
select date_format(t.endtime, '%M') as MonthName, sum(t.fare) as Revenue from tripdetails t
join bookings b on t.bookingid = b.bookingid
where b.status = 'Completed'
group by MonthName
order by MonthName;
```

	MonthName	Revenue
▶	April	199
	December	111
	February	210
	January	118
	July	187
	May	210
	November	92
	October	3245
	September	129

Insights: This query helps understand monthly revenue trends, allowing businesses to identify peak and low-earning months. It supports financial planning and forecasting by showing how much revenue comes from completed trips.

8. Write an SQL query to generate a performance summary of each driver, showing their average feedback rating, total number of trips completed, and total earnings generated. The result should group data by DriverID and sort by average rating in descending order.

```
select d.DriverID, d.Name, round(avg(f.Rating),2) as AvgRating,
    count(*) as TotalTrips,
    sum(t.Fare) as TotalEarnings
from drivers d
    join cabs c on d.DriverID = c.DriverID
    join bookings b on c.CabID = b.CabID
    join feedback f on b.BookingID = f.BookingID
    join tripdetails t on f.BookingID = t.BookingID
where b.Status = 'Completed'
group by d.DriverID, d.Name
order by AvgRating desc;
```

DriverID	Name	AvgRating	TotalTrips	TotalEarnings
101	John Driver	4.1	5	818
103	Kevin Road	4.05	4	855
104	Sandra Swift	3.8	5	975
102	Linda Miles	3.38	4	664
107	Tejas Kamble	3.25	2	221
105	Devendra Jacks	3.2	1	118
109	Gaurav Sawant	2.7	2	249
108	Rahul Kadam	2.6	3	282
106	Binod Bhatade	2.6	2	210
110	Sachin Bane	2.2	1	104

Insights: This query provides a comprehensive performance overview of each driver by combining ratings, completed trips, and earnings. It helps identify top-performing drivers who deliver good service and generate high revenue.

9. Write an SQL query to classify each trip as either a Short trip (less than 12 km) or a Long trip (12 km or more), and then calculate for each trip type the total number of trips and the total revenue generated.

```
select case when distancekm < 12 then 'Short' else 'Long' end as TripType,
    count(*) 'Number Of Trip',
    sum(fare) as TotalRevenue
from tripdetails
-- group by case when distancekm < 12 then 'Short' else 'Long' end;
group by TripType;
```

Result Grid | Filter Rows:

TripType	Number Of Trip	TotalRevenue
Long	15	3355
Short	24	2491

Insights: Classifying trips into Short and Long helps understand customer travel patterns and which trip type contributes more to revenue. This analysis highlights whether the business earns more from high-frequency short trips or high-value long trips, helping in strategic pricing decisions.

10. Write an SQL query to calculate the total revenue generated by each vehicle type. The revenue should only include trips from completed bookings. Use the Cabs, Bookings, and TripDetails tables, and group the results by vehicle type.

```
select c.VehicleType, sum(t.fare) as TotalRevenue from cabs c
join bookings b on c.cabid = b.cabid
join tripdetails t on b.BookingID = t.BookingID
where b.status = 'Completed'
group by c.VehicleType
order by TotalRevenue desc;
```

Result Grid | Filter Rows:

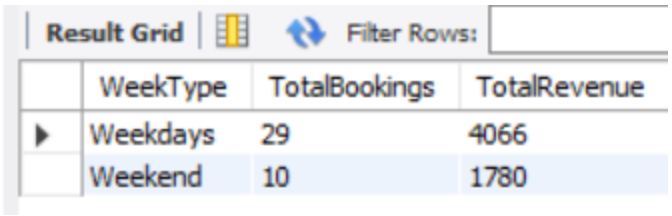
VehicleType	TotalRevenue
Sedan	1894
SUV	1748
Hyundai	459
Toyota	400

Insights: Vehicle types at the top of the list (e.g., SUVs or premium cars) contribute the most to overall revenue. Low-revenue vehicle types may need promotion, better utilization, or fare adjustments. High revenue indicates higher demand or frequent usage for certain vehicle types.

11. Write an SQL query to categorize each booking as either a Weekday or Weekend based on the booking date, and then calculate the following for each category:

1. **Total number of bookings, and**
2. **Total revenue earned (by summing the fare from TripDetails).**

```
select
    case
        when date_format(b.bookingdate, '%W') in ('Saturday', 'Sunday')
        then 'Weekend' else 'Weekdays' end as WeekType,
    count(*) as TotalBookings,
    sum(t.fare) as TotalRevenue from bookings b
join tripdetails t on b.bookingid = t.bookingid
group by WeekType;
```



	WeekType	TotalBookings	TotalRevenue
▶	Weekdays	29	4066
	Weekend	10	1780

Insights: Typically, weekends may show fewer but higher-fare trips due to leisure rides, while weekdays may have more trips but slightly lower fares. Comparing total revenue per category helps identify peak earning days, useful for pricing and promotional strategies.